QUESTORY

Run the city. Capture the story.

Final Project Report

TT2526HK3_CS426_24A – Android Mobile Development

24125023 Nguyen Trong Van Viet
24125078 Nguyen Hong Tan Tai
24125093 Nguyen Dinh Thien Loc (Team Leader)
24125107 Tran Le Anh Tuan

September 2026

Offline-first Android application, minimum API 24

Abstract

Questory is an offline-first Android application that combines recreational running, local discovery, location-aware photo quests, and visual storytelling. The initial content covers Nha Trang and Ho Chi Minh City. A user may choose a curated route or begin a free run, record a GPS trace, collect photo evidence at landmarks, review statistics and achievements, and turn the completed activity into an editable 1080 by 1920 story poster. Core use requires no account, server, or network connection. Data, retained photos, destination packs, and editable story documents remain under local control; anonymous online sharing is an optional adapter.

This report describes the problem, product decisions, domain model, layered architecture, implementation, reliability strategy, testing, team contribution, and technical self-assessment. It reflects the repository and automated results verified on 5 September 2026.

Keywords: Flutter, Android, offline-first, SQLite, GPS, photo quests, story editor, mobile application.

Contents

1 Project context and objectives

1.1 Background

Running applications are effective at measuring distance and pace, while travel applications are effective at describing places. These experiences are usually separate: a run becomes a graph with little cultural meaning, and a destination guide rarely turns exploration into an active, personal challenge. Questory connects the two. Movement provides the structure, landmarks provide motivation, and photographs become evidence and material for a visual diary.

The product was designed for an academic mobile-development project, but its scope is intentionally closer to a complete small application than a screen prototype. It has persistent local data, real location and camera integrations, error and permission states, navigation across the full journey, deterministic content, an editable export workflow, and an optional online extension.

1.2 Target users

- recreational runners who want a motivating route without performance pressure or a public leaderboard;
- visitors who prefer active exploration and need useful content even when mobile data is weak or unavailable;
- local residents who want a creative record of familiar places;
- privacy-conscious users who do not want an account for basic use; and
- student evaluators who need a repeatable, inspectable end-to-end flow.

1.3 Problem statement

The core problem is to record a credible run while helping the user notice the city and leave with something more expressive than a route screenshot. The solution must tolerate permission denial, disabled GPS, noisy points, temporary camera files, process interruption, missing network access, and image-export failure without losing the user's local activity.

1.4 Objectives and design principles

Questory's objectives are to provide useful bundled routes for two Vietnamese cities, support both structured and self-directed running, evaluate proximity quests fairly, preserve unfinished and completed work locally, and export a shareable portrait artifact. Five principles shaped the implementation:

1. **Offline is the default.** Network access cannot be a prerequisite for opening the app, recording, finishing, reviewing, or exporting.
2. **User action controls sensors.** Tracking begins only after an explicit explanation and confirmation.
3. **State is recoverable.** Active runs are checkpointed and retained photos move into application-controlled storage.
4. **The output remains editable.** Story Studio saves a serializable document rather than only a rendered image.

5. **Optional systems remain optional.** Remote sharing is behind an interface and disappears gracefully when not configured.

1.5 Originality and scope

Questory's distinctive element is the complete loop from place discovery to physical activity, evidence capture, personal achievement, and editable visual story. It is not a map viewer with decorative screens and not a camera gallery with a running label. The route, GPS track, quest evidence, summary, and story document share stable identities and survive navigation and application restart.

2 Product scope and user experience

2.1 Connected application surfaces

Surface	Purpose	Important behavior
Explore	Choose a launch city	Bundled packs; populated, empty, loading, and error states; no network dependency
Destination	Inspect a city and choose mode	Curated route cards, Free Run, pack version, landmarks, and quests
Route Details	Understand one route	Distance, duration, difficulty, safety notes, quest list, and explicit start
Run Tracker	Record the activity	Permission explanation; start, pause, resume, finish, discard, checkpoints, and quest actions
Quest Camera	Capture evidence	Proximity or fallback validation, retained photo, required caption, cancellation and camera-error recovery
Run Summary	Review the result	Distance, duration, pace, track, evidence, achievements, Story Studio, and optional online sharing
Journey	Reopen local work	Runs, editable stories, achievements, refresh, deletion confirmation, and empty/error states
Story Studio	Compose the visual diary	Original templates, transforms, layer order, undo/redo, local save/reopen, exact PNG export, share sheet

Table 1: Production application surfaces and their responsibilities.

The production navigation provides more than the minimum three to four connected screens. The bottom navigation keeps Explore and Journey understandable, while route selection, tracking, evidence capture, summary, and Story Studio appear as focused tasks in the natural order of use.

2.2 End-to-end user flow

The diagram and table describe the same production journey at two levels: the diagram shows how screens connect, while the table records the user action, application response, and recovery behavior at each stage. Importantly, neither curated routes nor Free Run depend on a server. Bundled destination content, SQLite persistence, device location, retained photographs, story editing,

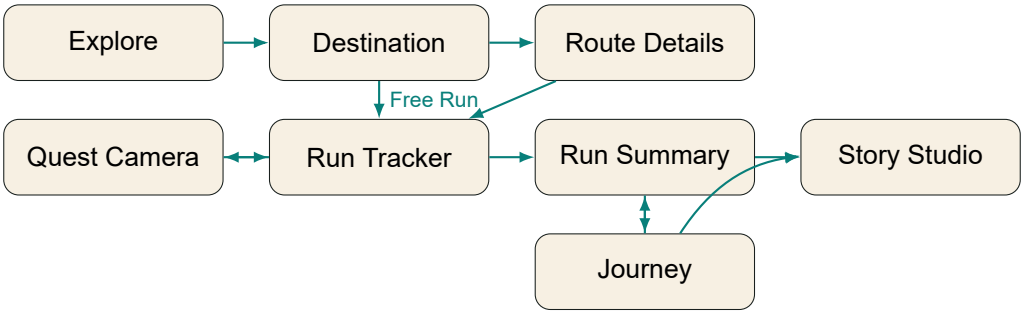

Figure 1: Questory's end-to-end production user flow.

Step	User action	Application response	Offline and recovery behavior
1	Launch Questory	Initialize local dependencies and open Explore with the Nha Trang and Ho Chi Minh City packs.	If initialization fails, show a retryable local-storage error instead of an empty or misleading home screen.
2	Choose a destination	Display bundled routes, landmarks, quests, estimates, and safety notes.	All launch content is packaged with the application; map tiles and a backend are not required.
3	Select a curated route or Free Run	Carry the selected route into tracking, or create an unrestricted session using the same recording pipeline.	The user can return without starting; no data is recorded before an explicit start.
4	Start tracking	Explain location use, check GPS service, request permission, and begin checkpointed recording after approval.	Denial and disabled GPS lead to recovery guidance; pause, resume, finish, and discard remain explicit.
5	Complete a photo quest	Validate proximity or a documented fallback, open the camera, require evidence and any necessary caption, then return to tracking.	Captured evidence is copied from temporary camera storage into app-controlled local storage.
6	Finish the run	Derive duration, distance, pace, quest results, and personal achievements; persist the summary; then offer Story Studio or Journey.	Invalid GPS jumps are filtered, zero-distance pace is handled, and the completed run is available without a network.
7	Build and share a story	Create or reopen an editable 1080 by 1920 project, arrange content, save it, export a deterministic PNG, and open Android sharing.	Editing and export are local; a failed share does not remove the saved project or completed run.
8	Revisit Journey	Browse prior runs and stories, reopen a summary or project, or deliberately delete a run.	Deletion removes related stories first, restores them if run deletion fails, and cleans retained photos only after consistency is confirmed.

Table 2: Detailed production user flow and its recovery paths.

and PNG export remain available offline. Only explicitly optional integrations, such as live sharing, may require connectivity.

2.3 Primary journey in detail

The user begins in Explore, where both launch packs are loaded from versioned JSON bundled with the application. Selecting a city reveals its routes, landmarks, quests, pack version, and offline status. A curated route provides an expected path, estimated distance, duration, and safety notes. Free Run keeps the same recording and storytelling pipeline without requiring a route.

Immediately before tracking, Questory explains why precise location is needed. After confirmation, the application checks the device service and requests permission. The tracker exposes start, pause, resume, finish, and discard actions. Quest cards display their radius and fallback rule. Capture opens the device camera only after proximity or explicit fallback confirmation.

Finishing produces a summary from the immutable session: active duration, distance, pace, track, landmarks, completed and skipped quests, retained evidence, and derived achievements. The result is stored in History and may become the source of a Story Studio project. The story can be edited, saved, reopened, exported as a 1080 by 1920 PNG, and passed to the Android share sheet.

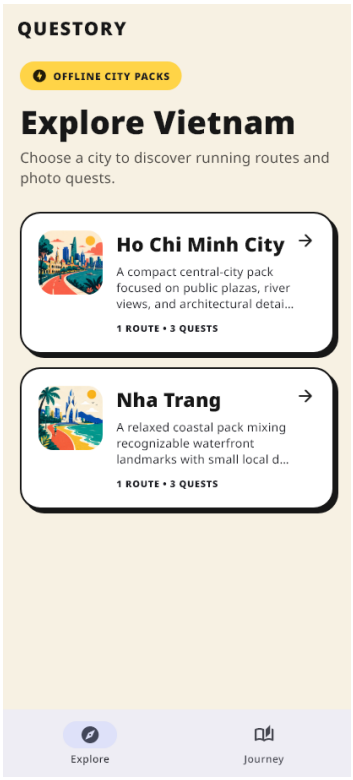

Figure 2: Explore uses bundled city packs and clearly communicates offline availability.

2.4 Interface direction

The visual language uses paper-like neutral surfaces, teal actions, coral and yellow accents, heavy display type, rounded cards, film frames, labels, and stickers. Solid colors avoid reproduction surprises and satisfy the product rule against gradients. Large controls support outdoor use; status labels and text do not rely on color alone.

3 Architecture and domain design

3.1 Layered structure

Questory uses feature-first modules with a shared domain and contract layer. The presentation layer depends on domain models and interfaces. Application classes coordinate state transitions and calculations. Data adapters implement SQLite, bundled JSON, filesystem, GPS, camera, export, Android sharing, and optional Supabase behavior. Dependency assembly occurs once at application startup.

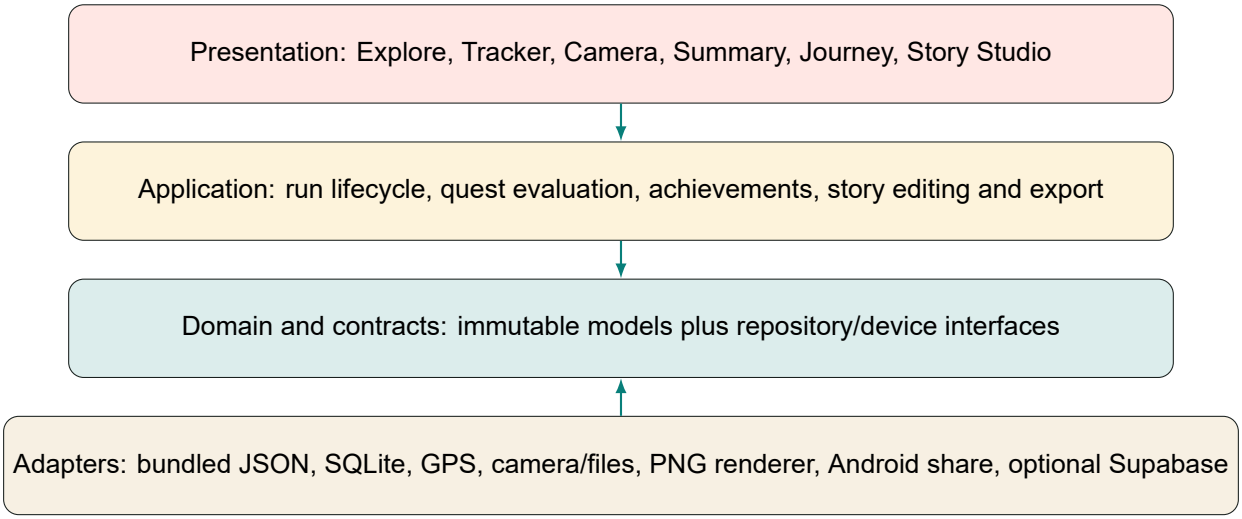

Figure 3: Layered dependency direction. Data adapters implement contracts; domain code does not depend on plugins.

This separation prevents the interface from opening databases or manipulating files directly. It also lets widget and application tests replace device services with deterministic fakes. Optional networking cannot change the outcome of a local run because it is represented by a nullable contract implementation rather than embedded in the core model.

3.2 Shared domain language

Model	Role
`GeoPoint`	Latitude, longitude, UTC timestamp, and optional accuracy or altitude
`DestinationPack`	City metadata, schema and pack version, routes, quests, and discovery points
`RoutePlan`	Expected polyline, distance, duration, landmarks, safety notes, and quest IDs
`Quest`	Prompt, point, radius, caption rule, evidence expectation, and fallback
`QuestEvidence`	Retained photo path, point, timestamp, caption, and quest identity
`RunSession`	Recoverable immutable snapshot of the active lifecycle and accumulated state
`RunSummary`	Final track, active duration, distance, pace inputs, evidence, and landmarks
`StoryDocument`	Serializable 1080 by 1920 canvas, ordered elements, transforms, and source run
`Achievement`	Stable criterion, progress, target, and optional UTC unlock timestamp

Table 3: Principal shared domain models.

Stable string identifiers connect records without exposing database row numbers. Timestamps are stored in UTC and displayed in local time. Distances stay in meters until presentation. Models

serialize to plain maps and lists so the database and tests can round-trip them without widget dependencies.

3.3 Dependency assembly

The production entry point first initializes Flutter, then opens the local database and constructs repositories and platform services. Only after this future succeeds does it render the main navigation. Startup loading and failure screens prevent a partially initialized UI. Supabase URL and anonymous key are read from compile-time definitions; when either is absent, the live-share service is simply unavailable and the complete offline application continues normally.

3.4 Offline-first boundary

The local path includes bundled destination content, SQLite records, application- controlled photos, metrics, achievements, Story Studio documents, export, and the Android share sheet. The remote path contains only anonymous route sharing. This boundary is architectural rather than cosmetic: a network failure cannot roll back or invalidate local state.

4 Feature implementation

4.1 Destinations and quests

Two versioned launch packs ship in application assets: Nha Trang and Ho Chi Minh City. Each pack contains city metadata, route plans, public landmarks, photo quests, fallback instructions, points of interest, and review metadata. The repository validates identifiers and relationships while decoding the JSON, so a malformed pack becomes an understandable loading failure rather than corrupting a run.

Quest eligibility is evaluated from the latest accepted location and the quest's radius. When GPS is unavailable or unreliable, the content-defined fallback may allow the user to confirm physical presence explicitly. A required caption is validated before evidence is committed. This balances location awareness with the practical limits of urban GPS and device testing.

4.2 Run lifecycle and location processing

The session lifecycle is represented explicitly as idle, active, paused, and finished states. Only active intervals contribute to duration. Pausing closes the current segment, while resuming opens a new segment. Finish derives a summary and clears the active checkpoint only after the final data has been saved.

Consecutive accepted points use the Haversine formula:

$$a = \sin^2(\Delta\varphi/2) + \cos(\varphi_1)\cos(\varphi_2)\sin^2(\Delta\lambda/2), \qquad d = 2R\arctan 2(\sqrt{a}, \sqrt{1-a}).$$

Implausible jumps are rejected using distance, elapsed time, and calculated speed. Non-increasing timestamps are invalid. Pace is unavailable for zero distance rather than represented by an infinite or misleading number. Calories are clearly presented as an estimate.

The Android location adapter requests high accuracy, applies a five-meter distance filter, and configures a foreground notification for an ongoing run. The manifest declares fine/coarse location and the location foreground-service type. Tracking still begins only after a user-visible action.

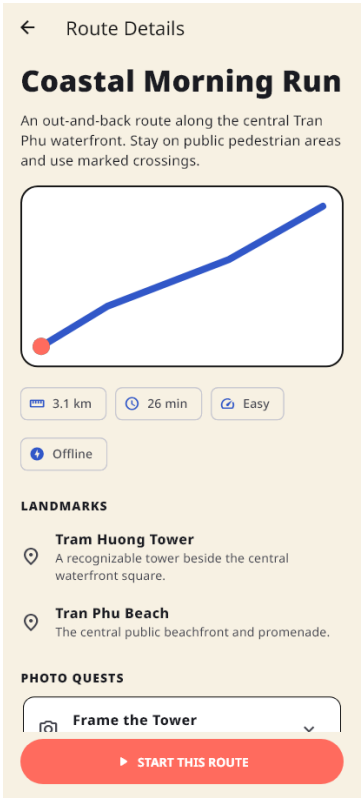

Figure 4: Route details present metrics, safety notes, landmarks, and photo quests before tracking begins.

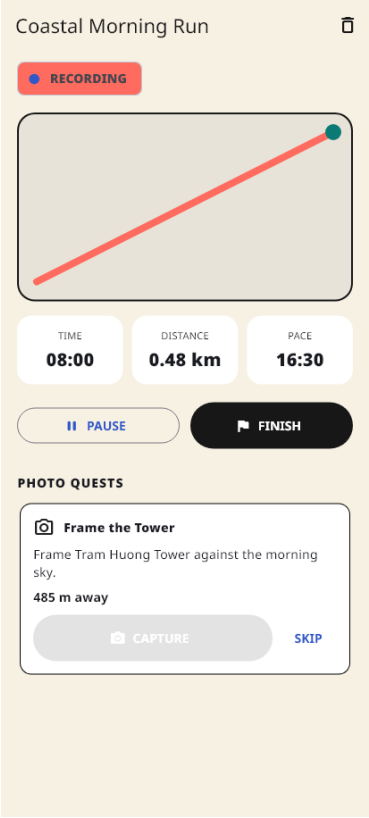

Figure 5: The active tracker keeps lifecycle controls, live metrics, and quest actions visible.

4.3 Camera evidence, Tracks, and retained media

Camera capture paths are temporary. After a valid capture and caption, the photo store copies the image into application-controlled support storage and the evidence record refers to that retained path. Cancellation, permission failure, and plugin errors return to a usable screen. History deletion removes dependent story documents before the run record and cleans retained evidence only after repository records are consistent.

The standalone Tracks experience adapts the camera interaction inherited from Nasr Al-Rahbi's MIT-licensed Flutter project *Lockit: Locket Clone App*, which provided the original capture, flash, camera-switching, preview, and local photo-history baseline. Questory retains the upstream copyright notice in `LICENSE` and lists the canonical repository in the References section. The team reworked that baseline for the production navigation: the camera is opened lazily from the central Tracks action, hardware and permission failures fall back safely, bundled photos support offline testing, captions and friend audiences are reviewed before acceptance, and mock accounts expose delivered versus locally queued sharing states. Accepted Tracks remain part of Questory's journey-and-story workflow rather than operating as a disconnected clone.

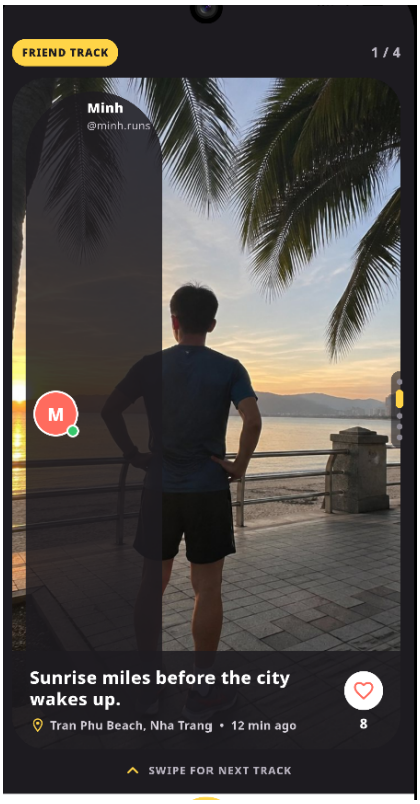

Figure 6: Tracks displays bundled friend photos with profile tags, captions, location labels, and reactions for the offline demo.

4.4 History and achievements

Journey combines completed runs, editable story projects, and personal achievements. Empty, loading, populated, and failure states are explicit. Run cards reopen summaries; story cards reopen their source project; destructive deletion requires confirmation. Achievement criteria are deterministic and personal, avoiding a public leaderboard or account requirement.

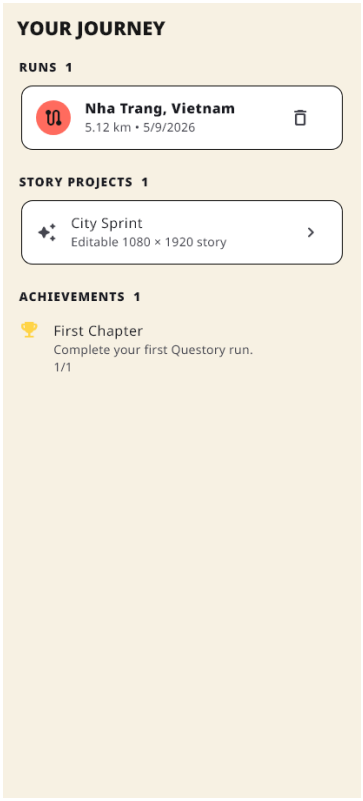

Figure 7: Journey preserves completed runs, editable story projects, and personal achievements.

4.5 Story Studio and export

Story Studio stores an ordered document with a fixed 1080 by 1920 canvas. Original templates provide starting compositions but remain ordinary editable documents. Elements carry their own type, content, style, layer order, and transform. Selection, move, scale, rotate, duplicate, delete, bring-forward, send-backward, undo, and redo operations produce new immutable documents.

Export renders from the saved document model rather than taking a screenshot of the editor. Selection borders, handles, and guides are not part of the rendered canvas. The output dimensions are checked before the PNG is offered to the Android share sheet. A failed renderer or share service produces recoverable feedback and does not destroy the project.

5 Implementation and technology choices

5.1 Technology stack

Flutter provides one strongly typed UI and domain codebase while retaining Android device integration. Material 3 supplies accessible interaction primitives, but the final surfaces use a custom solid-color identity. No large state-management framework was introduced; feature controllers and immutable snapshots keep the dependency surface small and inspectable.

5.2 Local persistence and recovery

SQLite is opened through a small database class with an explicit schema version. Repositories serialize active sessions, summaries, story documents, and achievements. Destination content

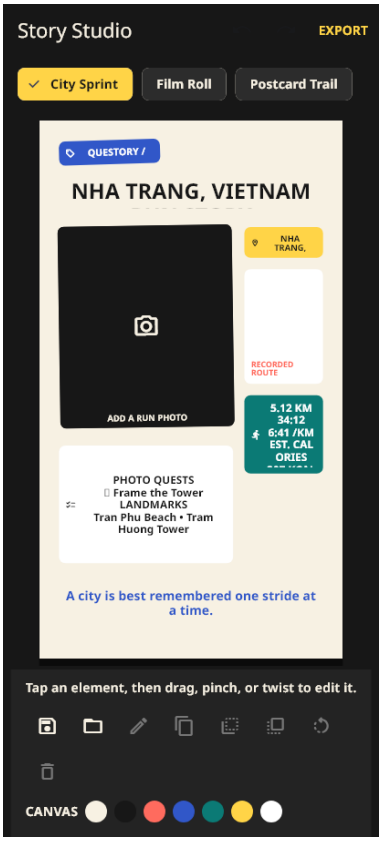

Figure 8: Story Studio turns run evidence into a layered, editable portrait composition.

Area	Technology	Reason
Application	Flutter and Dart	Typed cross-platform UI, mature Android tooling, immutable value models, and testable widgets
Local database	SQLite through `sqflite`	Transactional, versioned, offline application records on Android
Location	`geolocator`	Permission, service-state, position stream, and foreground-notification settings behind a contract
Camera	`image_picker`	Native camera handoff with a small adapter surface
Filesystem	`path_provider`, `path`	App-controlled support and cache locations without paths in widgets
Export and share	Flutter image codecs, `share_plus`	Deterministic PNG generation and the Android system share sheet
Optional remote	HTTP plus Supabase Edge Functions	Small REST boundary, anonymous expiring records, and no SDK in the domain
Testing	`flutter_test` and deterministic fakes	Logic, widget states, navigation, serialization, and failure behavior without devices or a server
Report	XeLaTeX and `latexmk`	Reproducible A4 output, modular source, stable tables, diagrams, references, and automated reruns

Table 4: Principal technologies and their roles.

stays in versioned JSON because it is bundled, reviewable, and deterministic. Photos are files because large binary payloads do not belong in normal relational rows.

The active run is checkpointed incrementally after meaningful state changes. When the application restarts, the main navigation checks for a checkpoint and offers Resume or Discard. Repository operations expose failures to presentation as recoverable states rather than allowing uncaught initialization or storage exceptions to leave a blank screen.

5.3 Consistency during deletion

Deleting a run is a cascade across story documents, the run summary, and retained photos. The orchestration removes related editable stories first, attempts the run deletion, restores stories if that record operation fails, and performs file cleanup only after repository state is consistent. Photo-cleanup failure is reported separately because the user record has already been deleted.

5.4 Optional online sharing

Anonymous link sharing is implemented behind `LiveShareService`. The client sends a deliberately limited payload to a Supabase Edge Function; the function validates the request, stores an expiring record, and returns a share URL. The anonymous application key may be compiled into the client, but the service-role key remains only in the function environment. Core operation never waits for this server. Before uploading, the UI names the shared coordinates, run time, captions, and evidence photos and requires explicit confirmation.

There are three valid run modes:

- **Offline production mode:** no server and no remote definitions;
- **Hosted sharing mode:** a deployed Supabase project and function;
- **Local sharing development:** Supabase CLI plus a Docker-compatible runtime, with the Android emulator addressing the host at `10.0.2.2` rather than `localhost`.

5.5 Platform scope

The release target is Android API 24 or later. The production entry point opens a mobile SQLite database and therefore must be run on an Android device or emulator. A browser entry point exists only as a focused Story Studio development harness. This distinction explains the reported `databaseFactory not initialized` screen on Windows/Edge: the wrong platform was selected before any normal user-flow screen could load. It is not evidence that a backend is missing.

6 Privacy, reliability, and quality assurance

6.1 Permission and privacy model

Questory requests only permissions connected to a visible action. Location is explained immediately before the system prompt; camera use begins from a quest action. Precise tracks, captions, and retained image paths are not intentionally logged. No account is required. Online sharing is opt-in, visible, time-limited, and restricted to a summarized payload rather than the editable local database. An explicit confirmation describes the precise route and media that will leave the device before any network request begins.

The app never promises that a public route is safe. Bundled notes remind the user to check current pedestrian conditions, weather, traffic, lighting, and local rules. Pause, finish, discard, and stop-sharing controls must remain discoverable.

6.2 Failure handling

The implementation treats expected failures as product states: unavailable city content, denied permission, disabled location service, camera cancellation, missing retained media, storage failure, export failure, remote failure, and empty History. Network failure cannot cancel or alter a local run. Zero-distance and missing-photo cases have safe representations instead of crashes.

6.3 Automated verification

Verification	Result	Meaning
`flutter analyze`	**Passed**	No static-analysis issues in the verified repository state
Complete Flutter test suite	**58 passed**	Domain, controller, persistence, widget, navigation, and export behaviors passed
Debug web compilation	**Passed**	Shared Flutter code compiles; this does not claim production browser persistence support
LaTeX build and log scan	**Passed**	Report compiles reproducibly; final page render is visually inspected after screenshots are inserted
Android minimum SDK	**API 24**	The application build explicitly supports the required minimum Android level

Table 5: Verified automated evidence as of 5 September 2026.

Tests cover distance calculations and invalid-jump filtering, active versus paused duration, zero-distance pace, quest proximity and fallback, achievement evaluation, persistence round-trips, story transforms and serialization, undo/redo, responsive screens, permission recovery, navigation, and export feedback. Platform plugins are wrapped so these behaviors are exercised with deterministic repositories, trackers, clocks, photo stores, renderers, and share services.

6.4 Visual and accessibility review

Core screens were rendered at a representative Android portrait size for the figures in this report. The review checks clipped text, overflow, contrast, control reachability, status meaning, empty/error copy, and separation between editor controls and exported story content. Real-device review remains important for system permission dialogs, camera return behavior, notification appearance, and continuous GPS under background conditions.

6.5 Engineering limitations

The product deliberately excludes turn-by-turn navigation, public rankings, weather, accounts, cloud synchronization, and Canva handoff. These would expand permissions, availability risks, and privacy obligations without improving the offline academic flow. The codebase also contains a focused Story Studio browser harness; it is development support and must not be confused with the Android production entry point.

7 Setup, configuration, and installation

7.1 Development prerequisites

The production application requires Flutter, an Android SDK, and either an Android emulator or a connected Android phone. Git and Java are provided by a normal Flutter Android toolchain. The optional local sharing server additionally requires a Docker-compatible runtime and the Supabase CLI. XeLaTeX and `latexmk` reproduce this report.

From the repository root, verify the toolchain and dependencies:

```
C:\dev\flutter\bin\flutter.bat doctor -v
C:\dev\flutter\bin\flutter.bat pub get
C:\dev\flutter\bin\flutter.bat devices
```

The device list must include an Android target. Start the complete application with that exact identifier:

```
C:\dev\flutter\bin\flutter.bat run -d <android-device-id>
```

No command must run before `main.dart` other than starting the Android emulator or connecting the phone. In particular, a backend is not needed for Explore, routes, Free Run, GPS tracking, quests, camera evidence, History, achievements, Story Studio, PNG export, or the Android share sheet.

7.2 Automated checks and Android build

```
C:\dev\flutter\bin\dart.bat format --output=none --set-exit-if-changed lib test
C:\dev\flutter\bin\flutter.bat analyze
C:\dev\flutter\bin\flutter.bat test
C:\dev\flutter\bin\flutter.bat build apk --debug
```

For the submission build, copy `android/key.properties.example` to the ignored `android/key.properties`. Use the private keystore path and credentials supplied to the team, then run:

```
C:\dev\flutter\bin\flutter.bat build apk --release
```

The verified output is placed at `apk/app-release.apk`. Signing credentials and the private keystore are never committed or packaged.

7.3 Optional hosted sharing

Only anonymous route-link sharing needs a server. After linking a Supabase project, apply the migration, configure the function's server-side environment, and deploy the function:

```
supabase login
supabase link --project-ref <project-ref>
supabase db push
supabase functions deploy share-run --no-verify-jwt
```

Then pass only the public project URL and anonymous key to Flutter:

```
C:\dev\flutter\bin\flutter.bat run -d <android-device-id> `
  --dart-define=SUPABASE_URL=https://<project-ref>.supabase.co `
  --dart-define=SUPABASE_ANON_KEY=<public-anon-key>
```

7.4 Optional local sharing server

For local function development, initialize once if the `supabase` directory does not exist, start the local stack, apply the migration, and serve the function:

```
supabase init
supabase start
supabase db reset --local
supabase functions serve share-run --no-verify-jwt
supabase status
```

On the standard Android emulator, use `http://10.0.2.2:54321` as `SUPABASE_URL`; a physical phone must use the development computer's reachable LAN address. Cleartext HTTP is allowed only in the debug Android manifest for this local workflow. Stop local services with `supabase stop`. Hosted deployment remains the appropriate way to show a link that another device can open.

7.5 Focused browser harness

Story Studio can be reviewed independently in a browser with:

```
C:\dev\flutter\bin\flutter.bat run -d edge `
  -t lib/features/story_studio/story_studio_demo.dart
```

This is a development harness for the editor only. It does not represent the complete production application or its SQLite/GPS/camera startup path.

8 Team organization and development process

8.1 Division of work

Student ID	Member	Primary module	Principal contribution evidence
24125023	Nguyen Trong Van Viet	Destinations and quests	Versioned destination schema; Nha Trang and Ho Chi Minh City packs; Explore, route detail, Free Run, proximity/fallback rules, and related tests
24125078	Nguyen Hong Tan Tai	Tracking and optional sharing	Run lifecycle, location stream, distance and jump filtering, duration and pace, checkpoints, foreground settings, and live-share adapter
24125093	Nguyen Dinh Thien Loc	Team leader; Story Studio, Tracks, and report	Product coordination, integration and release review; adaptation of the MIT-licensed Lockit camera baseline into the Tracks navigation, offline capture, mock friend and sharing flow; report organization and production; serializable story model, original templates, element operations, undo/redo, exact PNG renderer, Android sharing, and related tests
24125107	Tran Le Anh Tuan	Persistence, History, integration	SQLite schema and repositories, retained media, deletion cascade, achievements, Journey states, dependency assembly, recovery, Android configuration, and persistence tests

Table 6: Team ownership and observable deliverables.

Nguyen Dinh Thien Loc served as the team leader. In addition to owning Story Studio and export, Loc adapted and integrated the inherited Lockit camera into the production Tracks flow, implemented its offline-safe capture and mock social sharing states, coordinated the shared product direction, aligned work at integration points, reviewed release readiness, and organized and produced the final report and demonstration build. Leadership was collaborative: module owners retained responsibility for their deliverables, while cross-module decisions were made against the same offline-first architecture and acceptance criteria.

The repository history contains commits attributed to all four members. Module ownership identifies responsibility rather than exclusive access: each member could review other modules, while changes to domain contracts, dependency configuration, manifests, navigation, and the package specification were treated as integration hotspots.

8.2 Collaboration method

Work was divided against interfaces so a feature did not need to wait for a database, device, or another unfinished screen. Every principal contract has a deterministic fake. This made it possible to develop and test routes, tracking, Story Studio, History, and optional sharing independently before integration.

The shared engineering contract required contributors to inspect existing work, preserve unrelated changes, use stable domain language, keep remote behavior optional, format changed code, run proportionate tests, and state verification limits. Suggested feature branches and focused commits preserved visible member history. Secrets, private tracks, build caches, database files, and generated indexes were excluded from source control.

8.3 Integration decisions

The team selected immutable shared models before creating feature variants. Presentation depends on contracts, while SQLite and device plugins remain data adapters. Destination content is supplied through a repository contract; tracking persists through the run contract; Story Studio consumes a completed run summary; and History combines the repositories through dependency assembly.

This approach reduced merge coupling and made failure behavior testable. It also made architectural debt visible: cross-feature navigation and shared visual constants are integration responsibilities, not reasons for one feature to reach into another feature's private implementation.

9 Technical self-assessment

9.1 Functional completeness

The implemented journey exceeds a collection of disconnected screens: city content leads to route selection, tracking, quest evidence, a durable summary, History, and an editable export. Persistent storage and device integrations are part of the normal flow. The offline mode is the product's primary behavior, not a fallback demonstration state.

9.2 Technical quality

The strongest technical decisions are immutable domain snapshots, explicit lifecycle state, repository and device contracts, incremental checkpoints, app-controlled media, deterministic story ren-

Criterion	Evidence in the implementation	Assessment
Functional completeness	Eight connected surfaces, two modes, SQLite, GPS, camera, History, achievements, editable stories, PNG export, and system sharing	Complete cohesive core
Technical quality	Feature modules, immutable models, contracts, fakes, recovery, permission handling, filtering, deterministic export, and automated tests	Strong with contained integration debt
UI and responsiveness	Consistent visual system, Android-sized layout tests, discoverable actions, error/empty states, and portrait export preview	Coherent and suitable for outdoor use
Originality and complexity	Running plus travel discovery, proximity photo quests, recoverable activity state, and editable data-driven story output	Clear original product loop
Collaboration and reporting	Four ownership areas, visible commit authorship, shared engineering contract, comprehensive modular report, and reproducible setup	Traceable team process

Table 7: Evidence-based self-assessment against the project criteria.

dering, and optional networking. Tests concentrate on calculations, transitions, serialization, error paths, and responsive interface behavior. Android API 24 is explicit and dependencies are locked for reproducibility.

9.3 User experience and originality

The interface maintains a coherent outdoor journal identity and provides visible loading, empty, error, permission, active, paused, completed, and export states. Questory's originality lies in connecting running with local discovery and then transforming the same evidence into an editable story document. Personal achievements provide motivation without the privacy and moderation costs of a public leaderboard.

9.4 Responsible scope

Deferred features are deliberate: public accounts, rankings, turn-by-turn navigation, weather, synchronization, and third-party design handoff would add substantial backend, permission, safety, or licensing risk. The submitted scope instead emphasizes a reliable, inspectable, and complete local experience with one isolated optional server feature.

10 Conclusion

Questory demonstrates a complete offline-first Android application rather than a visual-only prototype. Its product loop begins with locally bundled Vietnamese city content, records a real activity through an explicit and recoverable state machine, evaluates location-aware photo quests, persists summaries and evidence, and ends with a deterministic editable story export. The architecture isolates device and server concerns behind contracts, allowing both reliable failure behavior and substantial automated verification.

The implementation's main strength is coherence: every major feature contributes to the same local journey and remains useful without an account or network. The optional Supabase adapter adds anonymous expiring links without weakening that invariant. The result satisfies the required

Android, navigation, persistence, device-integration, collaboration, and reporting scope while maintaining a distinctive identity centered on running, discovery, and personal storytelling.

References

1. Flutter, *Flutter documentation*, https://docs.flutter.dev/.
2. Android Developers, *Request location access at runtime*, https://developer.android.com/develop/sensors-and-location/location/permissions/runtime.
3. Android Developers, *Foreground service types*, https://developer.android.com/develop/background-work/services/fgs/service-types.
4. SQLite, *About SQLite*, https://www.sqlite.org/about.html.
5. Supabase, *Local development with schema migrations*, https://supabase.com/docs/guides/local-development/cli-workflows.
6. Supabase, *Edge Functions development environment*, https://supabase.com/docs/guides/functions/development-environment.
7. Nasr Al-Rahbi, *Lockit: Locket Clone App*, MIT-licensed Flutter source repository, https://github.com/abom-me/Lockit, accessed 5 September 2026.
8. Questory repository documentation, source code, versioned assets, tests, and Git history, reviewed 5 September 2026.